

[Pages](#) / [Core HPC Home](#) / [CoreHPC Access Primer](#)

CoreHPC Submission Examples

Created by [Torres, Luis](#), last updated on [Mar 10, 2026](#) • 5 minute read

 This guide is a work in progress.

- [SLURM Usage Guide for New Users](#)
 - [1. What Is SLURM?](#)
 - [2. Key Concepts](#)
 - [3. Submitting a Simple Job](#)
 - [4. Requesting CPU and Memory Resources](#)
 - [5. Running Jobs Requiring GPUs](#)
 - [6. Job Arrays](#)
 - [7. Running a Container as the Job \(Singularity/Apptainer\)](#)
 - [8. Running a Container Inside a Job Script](#)
 - [9. Using a GPU-Enabled Container](#)
 - [10. Interactive Jobs](#)
 - [11. Monitoring and Troubleshooting](#)
 - [12. Canceling Jobs](#)
 - [13. MPI and Multi-Node Jobs](#)
 - [Example: Running an MPI Job](#)
 - [14. Environment Modules](#)
 - [Common Commands](#)
 - [15. Cluster File Systems and Good Practices](#)
 - [16. Best Practices for Job Submissions](#)
 - [17. Example: Hybrid CPU + Container Workflow](#)
 - [18. Example: MPI Inside a Container](#)
 - [19. Troubleshooting Common Errors](#)
 - [20. Helpful SLURM Commands Summary](#)

SLURM Usage Guide for New Users

This guide introduces SLURM, a workload manager commonly used on computing clusters. It is intended for users with no prior experience on clusters. It includes basic concepts, how to submit jobs, how to request resources such as GPUs, how to use job arrays, and how to run containers.

1. What Is SLURM?

SLURM (Simple Linux Utility for Resource Management) is software that schedules and manages jobs on a compute cluster. Instead of running programs directly on login nodes, users submit jobs to SLURM, which allocates compute nodes and resources.

2. Key Concepts

- **Login Node:** Where you log in and prepare your work. Do not run heavy workloads here.
- **Compute Nodes:** Machines where jobs actually run (e.g., cnode01, gpu01).
- **Partitions:** Groups of nodes with similar capabilities (e.g., standard, gpu).
- **Job:** A script or command submitted to SLURM.
- **Job Script:** A file containing SLURM directives and commands.

- **Modules (env):** These are common software paths that can be added on-demand. The following commands can be used on the login node, and are available across all compute nodes.
 - 'module avail': What are the available modules.
 - 'module load <module>': Load <module>, where <module> is an item provided by 'module avail'.
 - 'module unload <module>': Unload <module>, sometimes certain modules will create conflicts, so the conflict modules must be unloaded.
 - 'module list': What are the modules that you have currently loaded.
 - 'module --help': What available commands does 'module' have?

Note: Using **srun** to launch your script or program within your batch (sbatch) script allows SLURM to better manage the run lifecycle of your job.

Any #SBATCH options can be overwritten from the command line as those options take precedence.

- A file called myscript has the option '#SBATCH --mem=4G'.
- e.g. 'sbatch --mem=8G myscript' will override the value in the script.

3. Submitting a Simple Job

Create a file, for example hello.sh:

hello.sh

```
#!/bin/bash
#SBATCH --job-name=hello_world
#SBATCH --output=hello.out

srun echo "Hello from SLURM job!!"
```

Submit it using:

- sbatch hello.sh

Check job status:

- squeue -u \$USER
- squeue

4. Requesting CPU and Memory Resources

Create a file, e.g. hello_gpu.sh with the example below, replace my_script.py with your actual script:

hello_gpu.sh

```
#!/bin/bash
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --time=02:00:00

srun python my_script.py
```

Submit it using:

- sbatch hello_gpu

Explanation:

- `--cpus-per-task=4`: reserves 4 CPU cores.
- `--mem=8G`: reserves 8 GB memory.
- `--time=2` hours: maximum runtime.

5. Running Jobs Requiring GPUs

Create the file `gpu_job.sh` with the example below:

gpu_job.sh

```
#!/bin/bash
#SBATCH --job-name=gpu_example
#SBATCH --output=gpu_example.out
#SBATCH --partition=gpu      # For POD nodes, use --partition=pod
#SBATCH --gres=gpu:1        # Request 1 GPU
#SBATCH --cpus-per-task=4   # This value is arbitrary and it can be higher.
#SBATCH --mem=16G
#SBATCH --time=04:00:00

module load cuda
srun python your_training_model.py
```

Execution:

- `sbatch gpu_job.sh`

Explanation:

- `--gres=gpu:1` requests 1 gpu resource.

6. Job Arrays

Job arrays allow submitting many similar tasks at once.

array_job.sh

```
#!/bin/bash
#SBATCH --job-name=array_example
#SBATCH --output=array_%A_%a.out
#SBATCH --array=1-100
#SBATCH --time=01:00:00
#SBATCH --mem=2G

srun python my_process_data.py my_input_${SLURM_ARRAY_TASK_ID}.txt
```

Explanation:

- `%A` = array job ID

- %a = task ID

For further information on Job Arrays, [please see the SchedMD Job Array documentation page](#).

7. Running a Container as the Job (Singularity/Apptainer)

Example using Apptainer (commonly available on clusters):

container_job.sh

```
#!/bin/bash
#SBATCH --cpus-per-task=8 #Arbitrary value.
#SBATCH --time=02:00:00
#SBATCH --mem=4G

module load apptainer
srun apptainer exec my_container.sif python my_script_inside_container.py
```

Note: you must have a prebuilt container sif file for this to work.

8. Running a Container *Inside* a Job Script

Use this if your main job environment is outside the container but you run certain tasks inside it.

container_job2.sh

```
#!/bin/bash
#SBATCH --job-name=mixed_job
#SBATCH --output=mixed_job.out
#SBATCH --partition=cpu
#SBATCH --time=03:00:00

module load apptainer

echo "Pre-processing outside container"
srun python preprocess.py

echo "Running containerized task"
srun apptainer exec my_container.sif python my_run_step.py
```

9. Using a GPU-Enabled Container

gpu_container.sh

```
#!/bin/bash
#SBATCH --job-name=gpu_container
```

```
#SBATCH --output=gpu_container.out
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --time=04:00:00

module load apptainer
module load cuda

srun apptainer exec --nv my_pytorch.sif python my_train_model.py
```

--nv makes GPU libraries visible inside the container.

10. Interactive Jobs

Interactive jobs give you a shell on a compute node.

```
salloc --partition=cpu --cpus-per-task=4 --mem=8G --time=01:00:00
```

Then run commands normally.

11. Monitoring and Troubleshooting

Check running/pending jobs:

- `squeue -u $USER`

View job details:

- `scontrol show job <jobid>`

View output logs: open the `.out` file created by the job.

12. Canceling Jobs

Cancel a single job:

- `scancel <jobid>`

Cancel all your jobs:

- `scancel -u $USER`

13. MPI and Multi-Node Jobs

MPI (Message Passing Interface) allows parallel jobs running across multiple nodes.

Example: Running an MPI Job

```
mpi_job.sh

#!/bin/bash
#SBATCH --job-name=mpi_example
```

```
#SBATCH --output=mpi_example.out
#SBATCH --partition=cpu
#SBATCH --nodes=2           # Request 2 nodes
#SBATCH --ntasks-per-node=8 # 8 MPI ranks per node
#SBATCH --time=02:00:00

module load openmpi/5.0.8
srun -n 4 mpi_program
```

14. Environment Modules

Clusters often use modules to load software environments.

Common Commands

List modules:

- `module avail`

Load a module:

- `module load python/3.10`

Show what modules are loaded:

- `module list`

15. Cluster File Systems and Good Practices

- **Home directory:** small, backed up, good for scripts/configs.
- **Project/Work directory:** high-capacity storage for large datasets and job outputs.
 - In CoreHPC there are two areas for this purpose: personal user scratch and group scratch. Scratch is designed to be temporary in nature.
 - `/mnt/scratch/user/${username}`
 - `/mnt/scratch/user/${FAC_Share}` (where FAC share is your CXyyyyyy_DSy share).
 - Please consult with the FAC team (<https://tiny.ucsf.edu/Gethpc>) if you do not have a FAC share and are interested in one.
- Avoid reading or writing excessively to network storage in tight loops; use local scratch if available:

```
/scratch/$USER/job_${SLURM_JOB_ID}
```

16. Best Practices for Job Submissions

- Test interactively before running large jobs.
- Request only the resources you need.
- Use job arrays for processing many similar files.
- Always specify reasonable `--time` limits.
- Use containers for reproducibility when possible.

17. Example: Hybrid CPU + Container Workflow

hybrid_cpu_contain.sh

```
#!/bin/bash
#SBATCH --job-name=hybrid_workflow
#SBATCH --output=hybrid_workflow.out
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=04:00:00

module load apptainer

# Step 1: Preprocess on host
srun python my_preprocess_data.py --input input.raw --output my_processed.json

# Step 2: Run containerized analysis
srun apptainer exec my_analysis.sif python my_analyze.py my_processed.json
```

18. Example: MPI Inside a Container

Some containers include MPI libraries compiled for host compatibility.

mpi_inside_contain.sh

```
#!/bin/bash
#SBATCH --job-name=mpi_container
#SBATCH --output=mpi_container.out
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --time=03:00:00

module load apptainer openmpi/5.0.8
srun apptainer exec my_mpi_container.sif my_mpi_program
```

19. Troubleshooting Common Errors

- **Job pending due to resource limits:**
 - Check with `squeue -j <jobid>` and `scontrol show job <jobid>`.
- **Out of memory (OOM):**
 - Increase `--mem` or optimize memory usage.
- **Module not found:**
 - Ensure correct module name; check `module avail`.
- **Container fails to run:**
 - Verify path, permissions, and required flags (`--nv` for GPUs).

20. Helpful SLURM Commands Summary

Command	Description
<code>sbatch script.sh</code>	Submit a job
<code>squeue -u \$USER</code>	List your jobs
<code>scancel <jobid></code>	Cancel a job
<code>sinfo</code>	View partitions and node states
<code>scontrol show job <jobid></code>	Detailed job info
<code>sacct -j <jobid></code>	Accounting info for finished jobs

No labels

Need help? Check out the [Wiki Help](#) space or the [FAQs](#) for answers to common issues.